

Отчёт - Чекпоинт 3: Интерфейс и развертывание

1. Описание функционала интерфейса

Реализован веб-интерфейс на базе Streamlit, обеспечивающий интуитивное взаимодействие пользователя с системой RAG. Интерфейс разделен на три основные области: боковую панель настроек, основную область ввода запросов и область отображения результатов.

1.1 Боковая панель настроек

Боковая панель предоставляет доступ к настройкам работы системы:

- **Режим работы:** Выбор одного из трех режимов поиска:
 - **Локальный RAG** — поиск по локальному корпусу статей arXiv
 - **Web-RAG** — поиск по интернету через Tavily API
 - **Гибридный локальный (DAT)** — комбинированный поиск с динамической настройкой весов
- **Топ-к документов:** Слайдер для настройки начального количества извлекаемых документов (диапазон: 3-10, значение по умолчанию: 5). Планировщик запросов может автоматически увеличить это значение на 3 при недостаточном качестве ответа

1.2 Основная область интерфейса

Центральная часть интерфейса содержит:

- **Заголовок и описание:** Логотип микроскопа, название “Science RAG” и краткое описание функциональности системы
- **Поле ввода вопроса:** Текстовое поле для ввода научного вопроса с поддержкой многострочного ввода
- **Кнопка “Получить ответ”:** Кнопка для запуска процесса поиска и генерации ответа

1.3 Область ответа

После выполнения запроса система отображает:

- **Процесс уточнения запроса:** При срабатывании планировщика автоматически показывается информация об итерациях уточнения, примененных стратегиях улучшения (переформулировка запроса, увеличение top-k на 3, или обе стратегии одновременно) и история всех запросов в раскрывающейся секции
- **Ответ:** Текст ответа, сгенерированный моделью Mistral на основе найденных документов
- **Метрики качества:** Средний и максимальный score релевантности найденных документов

1.4 Раздел источников

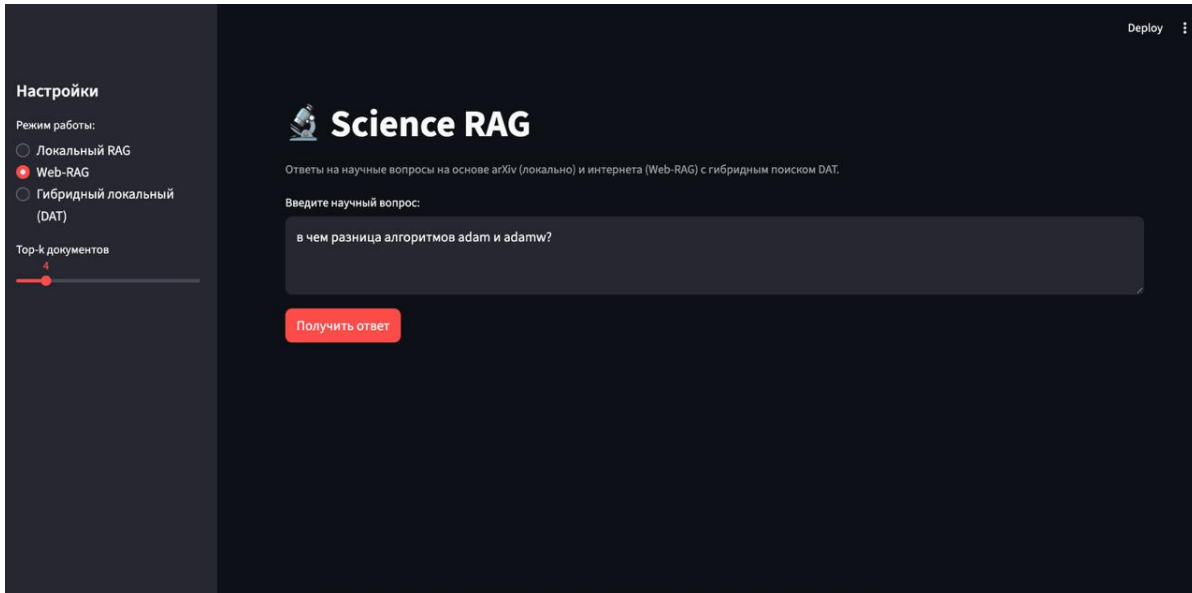
В нижней части интерфейса отображаются использованные источники:

- **Список источников:** Каждый источник представлен в раскрывающемся блоке с информацией:
 - ID документа и тип источника (local/web)
 - Score релевантности (от 0 до 1)
 - Заголовок документа (при наличии)
 - Ссылка на оригинальный источник (для web-источников)
 - Первые 1500 символов текста документа
- **Детали источника:** При раскрытии блока доступна полная информация о документе, включая математические формулы и подробные объяснения

2. Скриншоты интерфейса

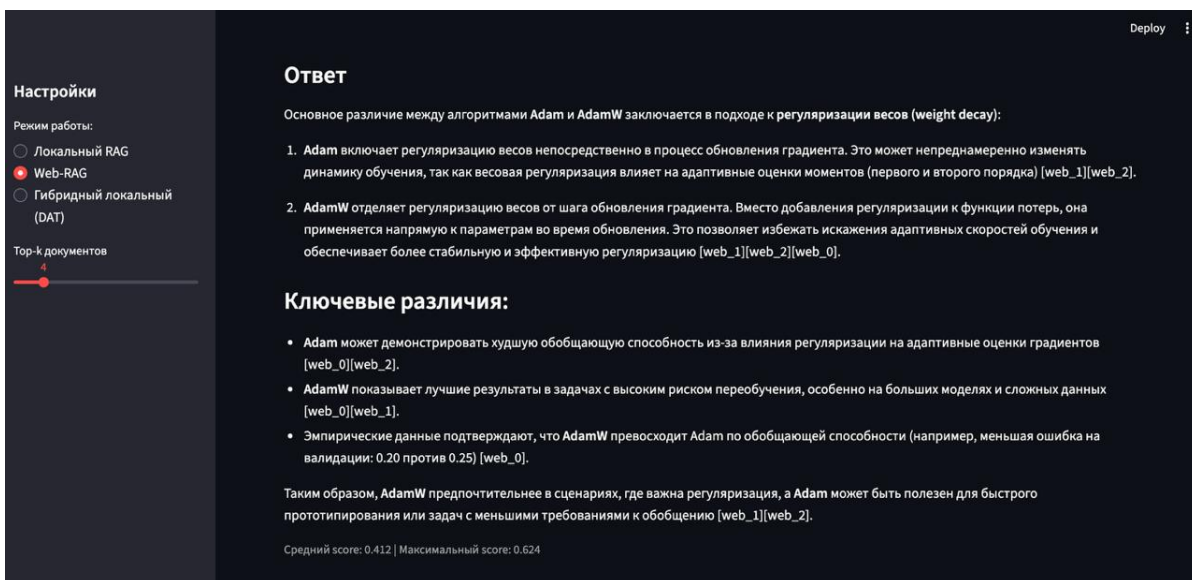
Скриншот 1: Главный интерфейс с настройками

На скриншоте представлена структура интерфейса: боковая панель с настройками режима работы (выбран режим “Web-RAG”) и параметра top-k документов (значение 4), основная область с полем ввода вопроса и примером запроса “в чем разница алгоритмов adam и adamw?”.



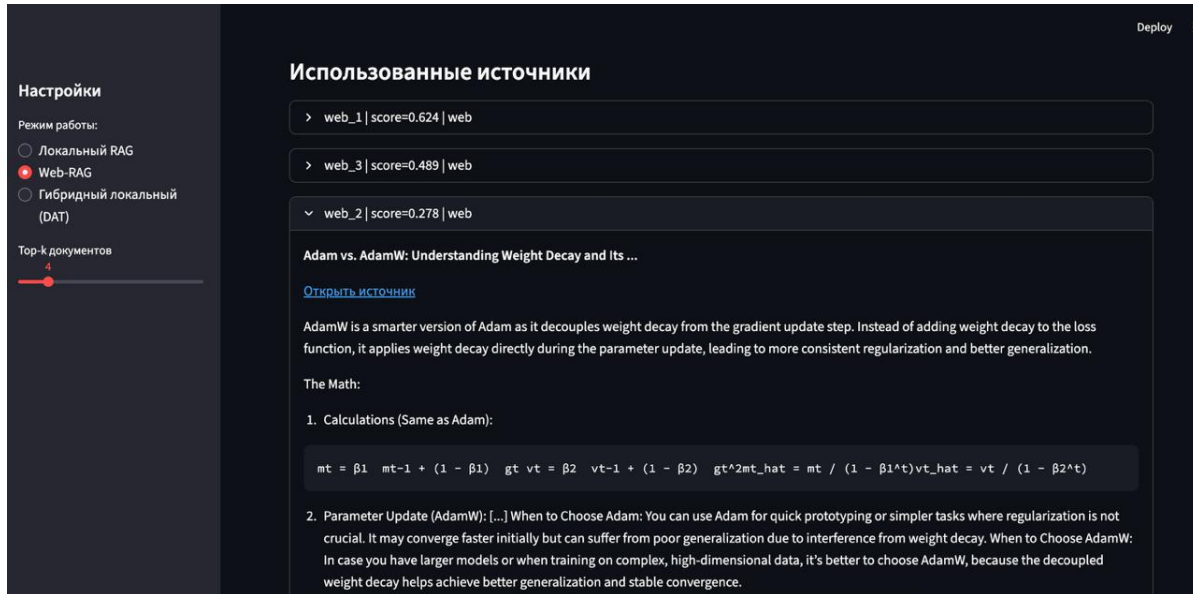
Скриншот 2: Ответ с метриками качества

На скриншоте показан результат работы системы: подробный ответ на русском языке с объяснением различий между алгоритмами Adam и AdamW, ссылками на источники [web_0], [web_1], [web_2], и метриками качества (средний score: 0.412, максимальный score: 0.624).



Скриншот 3: Развернутые источники

На скриншоте показана детальная информация об источниках: при раскрытии блока отображается полное содержание документа web_2 с заголовком статьи “Adam vs. AdamW: Understanding Weight Decay and Its...”, ссылкой на оригинальный источник, описанием, математическими формулами и рекомендациями по выбору алгоритма.



3. Инструкция по запуску

3.1 Запуск Streamlit-приложения

Для запуска приложения необходимо установить переменные окружения с API ключами:

```
export MISTRAL_API_KEY="..."
export TAVILY_API_KEY="..."
```

```
streamlit run app/streamlit_app.py
```

После запуска приложение будет доступно по адресу <http://localhost:8501>. В интерфейсе доступны три режима работы:

- **Local RAG** — поиск по локальному корпусу arXiv
- **Hybrid RAG (DAT)** — гибридный поиск с динамической настройкой весов
- **Web-RAG** — поиск по интернету через Tavily API

3.2 Запуск с Docker

Система поддерживает развертывание через Docker Compose для упрощения установки и запуска.

Требования

- Установленные Docker и Docker Compose
- API ключи для Mistral и Tavily

Подготовка

1. Создайте файл `.env` в корне проекта со следующим содержимым:

```
MISTRAL_API_KEY=your_mistral_api_key_here  
TAVILY_API_KEY=your_tavily_api_key_here
```

2. Убедитесь, что данные подготовлены: индексы FAISS и BM25 должны быть созданы (см. раздел 4 в отчете чекпоинта 1).

Запуск Для сборки и запуска контейнера выполните:

```
docker-compose up --build
```

После успешного запуска приложение будет доступно по адресу `http://localhost:8501`

4. Обработка ошибок и сообщения пользователю

В Streamlit-приложении реализована комплексная система обработки ошибок с понятными сообщениями на русском языке. Обработка ошибок разделена на три основных уровня:

4.1 Обработка ошибок инициализации pipeline

При инициализации pipeline выполняется проверка конфигурации и наличия необходимых ресурсов:

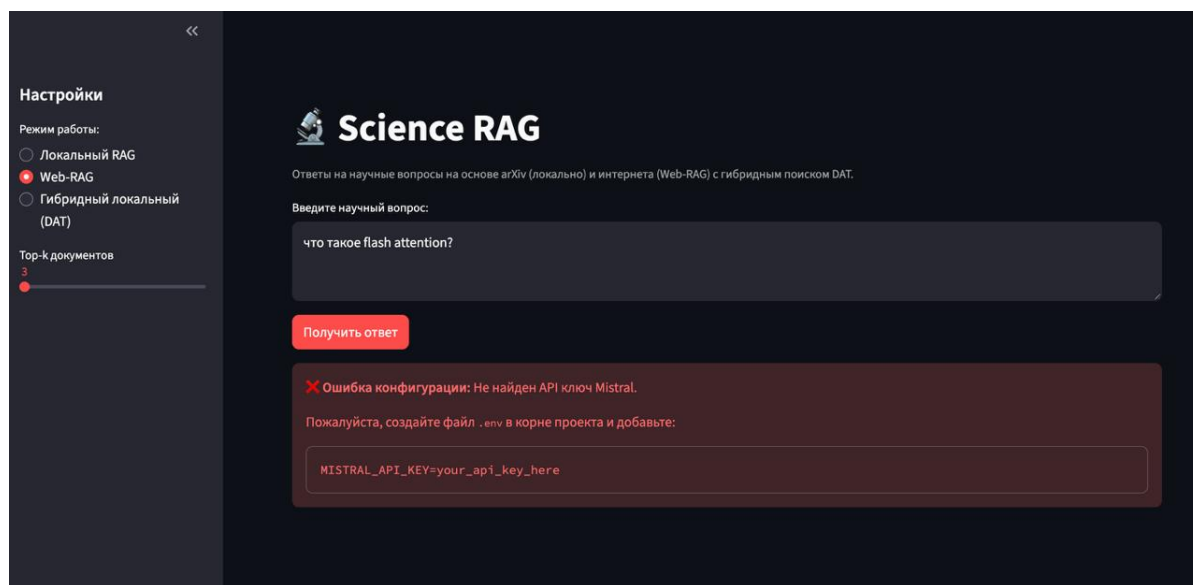
- **Проверка API ключей:** Система проверяет наличие переменной окружения `MISTRAL_API_KEY` и выводит понятное сообщение с инструкцией по созданию файла `.env`, если ключ отсутствует
- **Проверка файлов данных:** Перед загрузкой pipeline проверяется наличие всех необходимых файлов:
 - `data/chunks/local/chunks.jsonl`
 - `data/faiss/local_dense.index`
 - `data/faiss/local_dense_meta.json`
 - `data/chunks/local/bm25_index.json`
 - `data/chunks/local/bm25_meta.json`
- **Обработка ошибок чтения файлов:** При отсутствии файлов выводится список отсутствующих файлов с рекомендацией проверить подготовку данных
- **Обработка ошибок формата данных:** При обнаружении некорректного JSON в файлах данных система сообщает о возможном повреждении файлов и рекомендует пересоздать индексы

4.2 Обработка ошибок при обработке запросов

При выполнении запросов система обрабатывает различные типы ошибок с конкретными сообщениями:

- **Ошибки API:** Система распознает проблемы с внешними API (Mistral, Tavily) и выводит сообщение с возможными причинами:
 - Неверный или отсутствующий API ключ
 - Превышен лимит запросов
 - Проблемы с сетью
 - Рекомендация проверить настройки в файле `.env`

На скриншоте ниже показан пример обработки ошибки конфигурации при отсутствии API ключа Mistral:



- **Ошибки сети:** При проблемах с подключением к интернету выводится сообщение с рекомендацией проверить подключение
- **Ошибки индексов:** При проблемах с загрузкой или использованием индексов поиска (FAISS, BM25) система сообщает о возможном повреждении индексов и рекомендует их пересоздание
- **Ошибки эмбедингов:** При проблемах с генерацией эмбедингов выводится сообщение с возможными причинами:
 - Проблема с моделью эмбедингов
 - Недостаточно памяти
 - Проблемы с загрузкой модели
- **Общие ошибки:** Для всех остальных ошибок выводится тип ошибки, сообщение и рекомендации по решению проблемы

Для всех типов ошибок доступна опция показа детального traceback через чекбокс “Показать детали ошибки”, что упрощает отладку для разработчиков.

4.3 Обработка ошибок при отображении результатов

Система также обрабатывает ошибки при отображении результатов:

- **Отображение истории запросов:** При ошибках отображения истории запросов выводится предупреждение без остановки работы приложения
- **Отображение ответа:** Проверяется наличие ответа, при его отсутствии выводится предупреждение о возможных причинах
- **Вычисление метрик:** При ошибках вычисления метрик качества выводится предупреждение, при отсутствии источников — информационное сообщение
- **Отображение источников:** Каждый источник обрабатывается индивидуально, при ошибке отображения одного источника остальные продолжают отображаться; при отсутствии источников выводится рекомендация изменить запрос или увеличить top-k

Все сообщения об ошибках оформлены с использованием эмодзи для лучшей визуальной идентификации (❌ для ошибок, ⚠️ для предупреждений, ℹ️ для информационных сообщений) и содержат конкретные рекомендации по решению проблем.

5. Финальные метрики качества системы

Результаты тестирования всех трёх retrievers на валидационной выборке из 859 вопросов:

Retriever	R@1	R@3	R@5
BM25	0.356	0.471	0.508
Dense	0.161	0.263	0.312
Hybrid + DAT	0.232	0.346	0.455

Анализ результатов

BM25 демонстрирует наилучшие результаты по всем метрикам: - Recall@1 = 0.356: в 35.6% случаев релевантный документ находится на первой позиции - Recall@5 = 0.508: более половины релевантных документов присутствуют в топ-5 результатах - Эффективность BM25 объясняется точным совпадением ключевых слов на терминологических запросах

Dense retrieval показывает более низкие результаты: - Recall@1 = 0.161: семантический поиск менее точен для точного попадания на первую позицию - Recall@5 = 0.312: семантический поиск находит релевантные документы, но с меньшей точностью позиционирования

Hybrid + DAT занимает промежуточное положение: - Recall@1 = 0.232: превосходит Dense, но уступает BM25 - Recall@5 = 0.455: близок к результатам BM25, что подтверждает эффективность комбинирования методов - Динамическая настройка весов α позволяет адаптировать поиск под каждый запрос, однако требует дальнейшей оптимизации

Выводы: - BM25 остается сильным baseline для данного типа запросов - Hybrid+DAT демонстрирует потенциал для улучшения через более точную настройку весов (например, LLM-based scoring α) - Система готова к использованию в production с учетом ограничений каждого метода поиска